

Database Management Guide — Nucleus AI

Database Architecture

Nucleus AI uses two databases:

1. **PostgreSQL** — Primary relational database (users, campaigns, spend, conversions)
 2. **Qdrant** — Vector database (Context Vault embeddings)
-

Migrations (Alembic)

Setup

```
cd backend
alembic init alembic
```

Update `alembic/env.py` to use async engine:

```
from db.database import Base
from db.models import * # noqa: import all models
target_metadata = Base.metadata
```

Common Operations

```
# Create new migration
alembic revision --autogenerate -m "add campaign budget field"

# Apply all pending migrations
alembic upgrade head

# Rollback one migration
alembic downgrade -1

# View migration history
alembic history

# View current revision
alembic current
```

Production Migrations

```
# Run inside the container
docker compose -f docker-compose.prod.yml exec backend alembic upgrade head

# Or from CI/CD
ssh user@server "cd /opt/nucleus-ai && docker compose -f docker-compose.prod.yml exec -T backend alembic upgrade head"
```

Backup & Restore

Automated Backups

```
# Set up daily backups
chmod +x scripts/deployment/backup.sh

# Add to crontab (daily at 2 AM)
(crontab -l 2>/dev/null; echo "0 2 * * * /opt/nucleus-ai/scripts/deployment/
backup.sh") | crontab -
```

Manual Backup

```
./scripts/deployment/backup.sh /path/to/backup/dir
```

Restore

```
./scripts/deployment/restore.sh /opt/backups/nucleus-ai/db_20260413_020000.sql.gz
```

Connection Pooling

Configured in `backend/db/database.py` :

```
engine = create_async_engine(
    settings.DATABASE_URL,
    echo=False,           # Disable SQL logging in production
    pool_pre_ping=True,   # Validate connections before use
    pool_size=20,         # Base pool size
    max_overflow=30,      # Extra connections when pool is full
    pool_timeout=30,      # Seconds to wait for a connection
    pool_recycle=3600,    # Recycle connections after 1 hour
)
```

Recommended Pool Sizes

Deployment Size	pool_size	max_overflow	Total Max
Small (1 worker)	10	10	20
Medium (2-4 workers)	20	30	50
Large (4+ workers)	30	50	80

Rule: Total connections across all workers should stay under PostgreSQL's `max_connections` (default: 100).

Scaling Considerations

Vertical Scaling

- Increase RDS/Cloud SQL instance size
- Adjust `shared_buffers` and `effective_cache_size`

Read Replicas

- Use read replicas for analytics queries (Attribution Engine)
- Route write operations to primary, reads to replicas

Partitioning (for high-volume tables)

```
-- Partition spend_logs by month for faster queries
CREATE TABLE spend_logs (
  id UUID DEFAULT gen_random_uuid(),
  campaign_id UUID NOT NULL,
  amount FLOAT NOT NULL,
  channel VARCHAR(255),
  created_at TIMESTAMP DEFAULT NOW()
) PARTITION BY RANGE (created_at);

CREATE TABLE spend_logs_2026_q1 PARTITION OF spend_logs
  FOR VALUES FROM ('2026-01-01') TO ('2026-04-01');
```

Monitoring Queries

```
-- Active connections
SELECT count(*) FROM pg_stat_activity WHERE datname = 'nucleus';

-- Long-running queries (> 5s)
SELECT pid, now() - pg_stat_activity.query_start AS duration, query
FROM pg_stat_activity
WHERE (now() - pg_stat_activity.query_start) > interval '5 seconds'
AND state != 'idle';

-- Table sizes
SELECT relname, pg_size_pretty(pg_total_relation_size(relid))
FROM pg_catalog.pg_statio_user_tables
ORDER BY pg_total_relation_size(relid) DESC;

-- Index usage
SELECT relname, indexrelname, idx_scan, idx_tup_read
FROM pg_stat_user_indexes
ORDER BY idx_scan DESC;
```